

异构算力跨架构技术与开源生态

赵奇慧

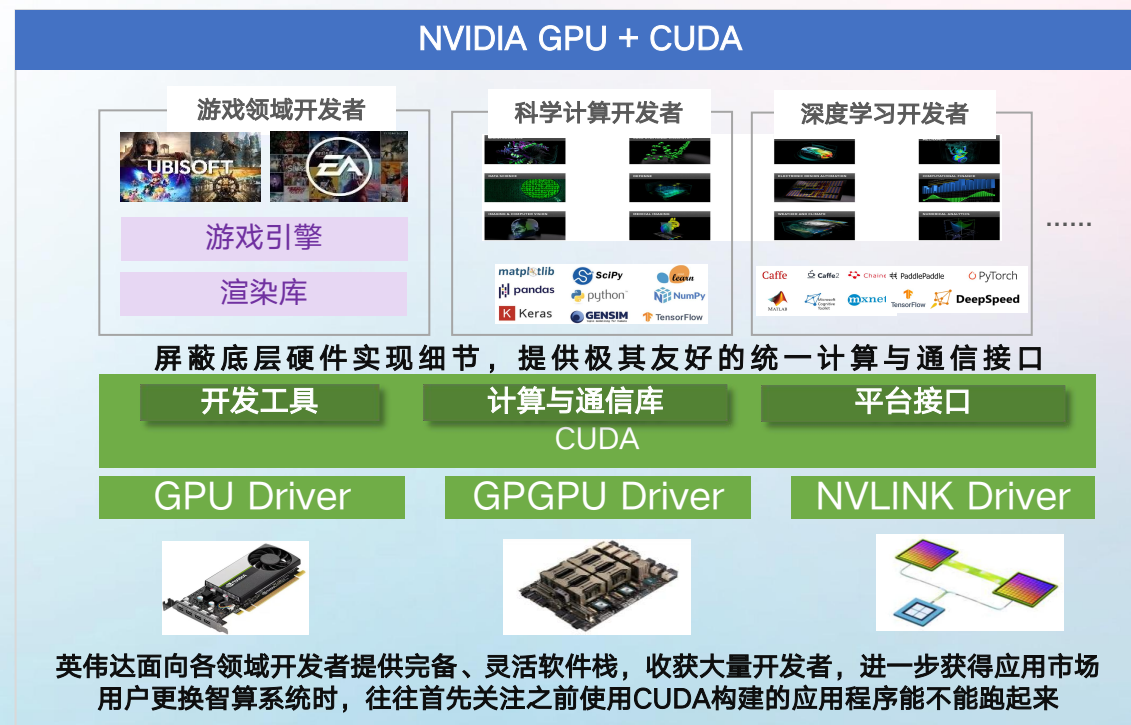
中国移动研究院

2025/11/15



系统的核心竞争力在于软件生态

我们常常讨论更低延时、更高带宽、更大规模等系统指标，
但用户在购买系统产品的时候，需要考虑的因素并不仅仅是硬件性能，更是软件生态

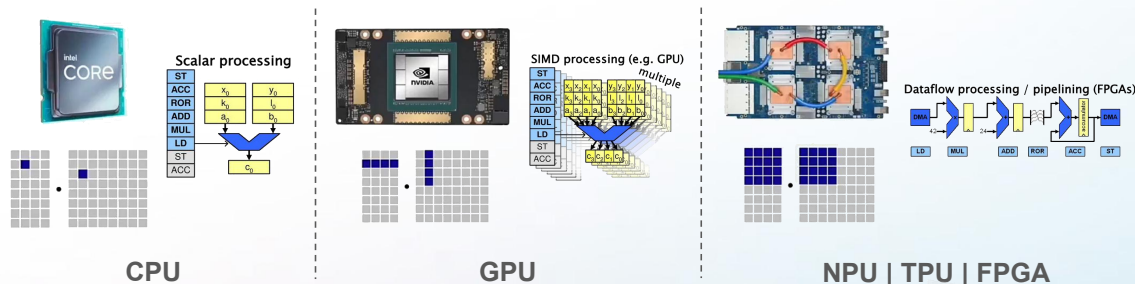


开发人员需要运用硬件厂商所提供的运行环境及与之匹配的开发工具链进行软件研发，开发工具链越友好、运行环境越通用，
开发者社区越壮大，随之软件生态越丰富、用户粘性越强、市场规模越大

智算系统呈现异构化发展趋势

用户选用智算硬件需平衡计算能力（专用、通用）、计算性能、成本（购买成本、运维成本、时间成本等），各异的用户需求将导致计算架构保持多元化发展趋势，异构芯片共存成为常态

计算架构因用户需求各异将保持多元化发展趋势



效能比
通用性

性能

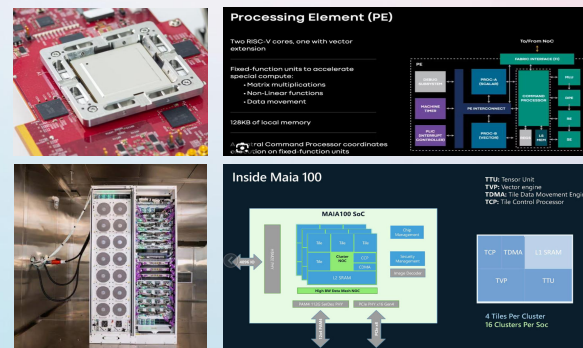
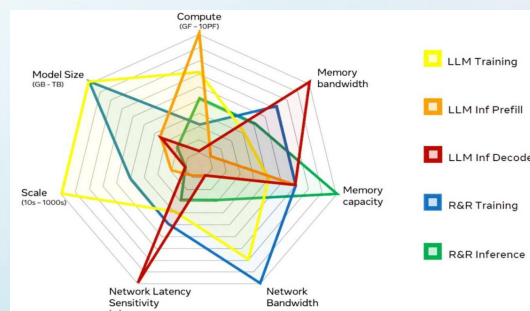
HARDWARE	256 TOKENS ↑	1024 TOKENS ↑
CPU	23.21 TOKS/S	19.63 TOKS/S
GPU	107.00 TOKS/S	107.24 TOKS/S
FPGA	57.11 TOKS/S	57.11 TOKS/S

功耗

HARDWARE	256 TOKENS ↓	1024 TOKENS ↓
CPU	42.5 W	42.5 W
GPU	126.9 W	130.6 W
FPGA	9 W	9 W

多种异构芯片共存的系统形态

针对自身智算推理业务负载（特别是大模型负载），MetaAI、微软均开始定制芯片，芯片内本身由“CPU+NPU+CustomEngine”多种不同的计算内核构成，同时系统内均混布有多厂商、多类型AI芯片。



集群形态

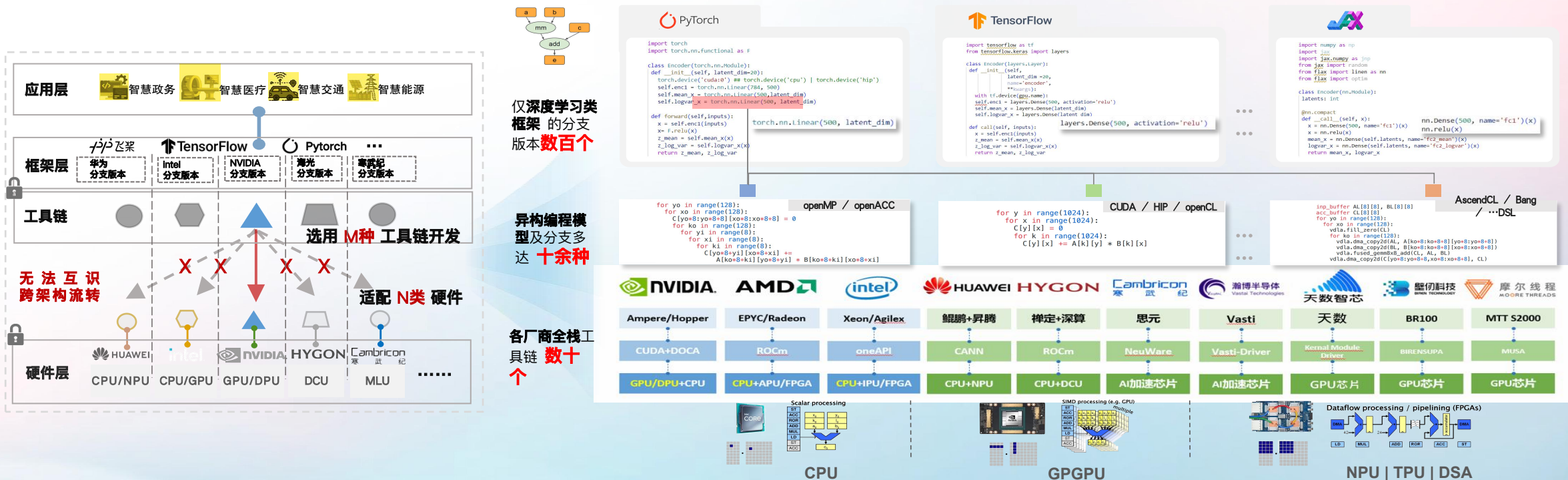
多厂商、多类型AI芯片互联、协同运用

边缘形态

多厂商、多类型AI芯片多规格一体组合

多元异构算力协同面临智算竖井生态挑战

算力界的Android尚未出现。各硬件厂商围绕自身硬件特性构造“AI框架+工具链+硬件”端到端竖井智算生态，互不兼容，上层智算应用与特定硬件锁定，难以跨架构动态迁移、优化适配，制约基础设施整体计算效能释放



问题一：应用开发成本高

编程模型、工具链各不相同，致使同一应用需多次开发，开发繁复/维护成本高

问题二：资源规划难、利用率低

应用难以迁移部署，使资源合理规划难、利用率低，难以释放整体效能

问题三：新硬件应用生态拓展难

应用生态碎片化发展，新硬件难以基于自身软硬件拓展应用，良性生态难以建立

业界跨架构技术路线分析

屏蔽多类异构计算系统差异、涉及多类软硬件“交叉”技术领域，是典型的复杂系统工程。当前产学研各界开展了广泛的研究，在框架、基础软件、硬件等不同层级进行了多路径的探索，路线二在业务通用性和实现可行性上具备优势

框架层：

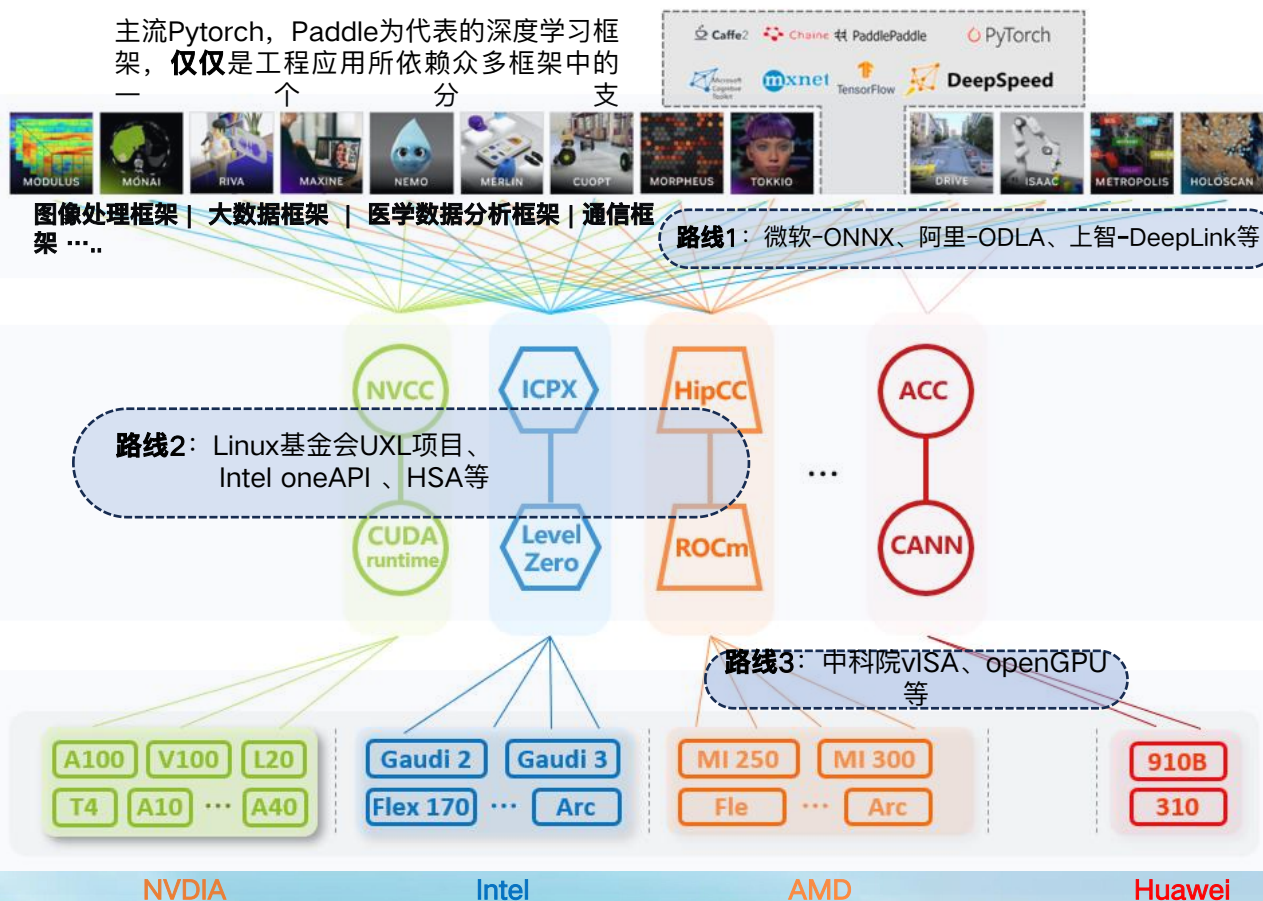
- 与行业及领域强相关，接口各异、规模 **数万**

基础软件层：

- 各自建立系统抽象
- 形成开发语言、编译、链接一体的 **十余** 个基础软件

硬件层：

- 各有特色及性能侧重
- 不同厂商、不同代际，架构种类 **数十** 个



路线一：统一框架或领域算法接口

算子与功能接口的**标准化**，形成单一领域算法接口，但需依赖厂商工具链进行多次开发，有明显领域局限性，**通用性较差**

路线二：统一基础软件

异构系统模型及机制统一，形成统一编程和系统接口，应用仅需一次开发，即可实现与各厂商运行时适配对接，**通用性较高**

路线三：统一指令集

硬件指令集标准统一，形成各厂商工具链共同遵循的**统一虚拟指令集**，但需要厂商深度更改硬件架构，**通用性强但实现难度较大**

弱 易

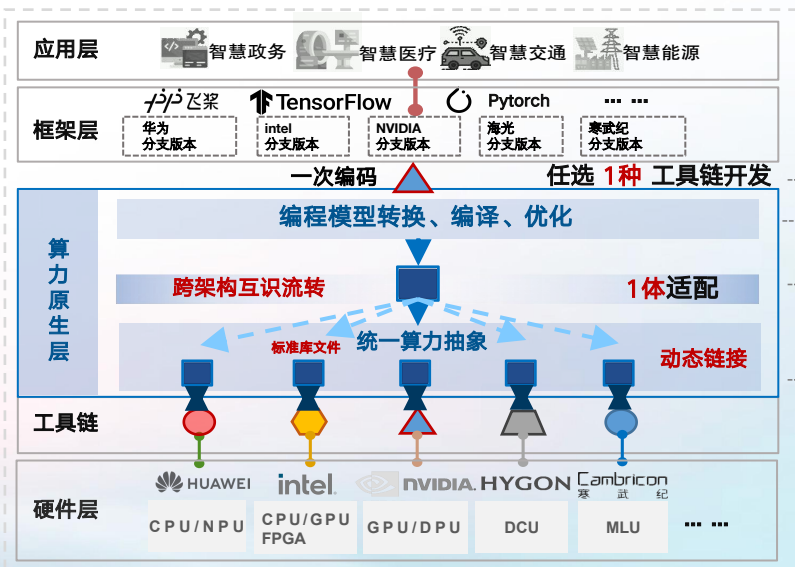
通用性、部署灵活性、可持续性
工程实现性、产业推动难

强 难

算力原生新型技术机制屏蔽异构差异

算力原生核心理念是在框架与底层硬件之间构建相对通用的统一的算力抽象机制与接口，向上为用户提供统一编程入口，向下使能底层硬件一体适配，通过四大关键技术，实现应用一次开发、跨芯迁移部署，任务式自适应执行的目标

算力原生：编译链接解耦，编译仅生成中间元语言文件，部署时根据所调度的硬件进行动态链接。



使能应用1+1次适配，实现跨架构迁移

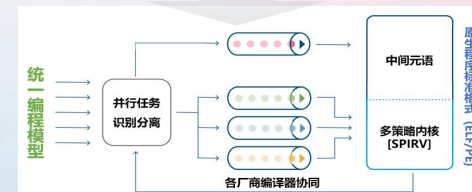
关键技术一：统一编程模型技术

对混合异构系统**数据、内核、并行机制的统一编程语言范式**；突破**源源转换**技术，使其他编程模型向统一编程模型进行代码级迁移转换。

```
1 #include <gcc/rycl.hpp>
2 #include <iostream>
3
4 using namespace rycl;
5
6 int main() {
7     constexpr int size=10;
8     buffer<int> B { range<size>() };
9
10    // Create queue on any available device
11    queue Q;
12    Q.submit([&(handler& h) {
13        accessor A(B, B);
14        h.parallel_for(size - [&](const id& i) {
15            A[i] = 10;
16        });
17    });
18    // Obtain access to buffer on the host
19    // Will wait for device kernel to execute to generate data
20    host_accessor AB;
21    for (int i = 0; i < size; i++)
22        std::cout << "data[" << i << "] = " << A[i] << "\n";
23
24    return 0;
25 }
```

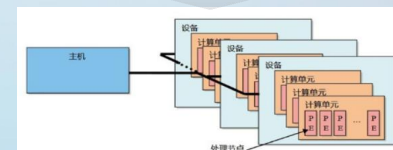
关键技术二：跨架构编译优化技术

研究可融入各厂商设备编译器的**开放式编译协同机制**，图、算**融合编译优化方法**，形成**个性化执行策略**，生成互识的算力原生标准格式程序，保证跨架构的同时最大化兼顾运行性能。



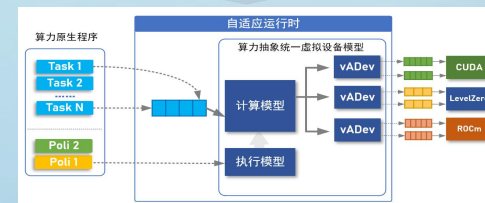
关键技术三：算力抽象技术

为多类异构加速硬件建立**统一设备逻辑模型、内存模型、执行模型**等，形成**虚拟算力设备机制**及相匹配的系统接口与映射标准。



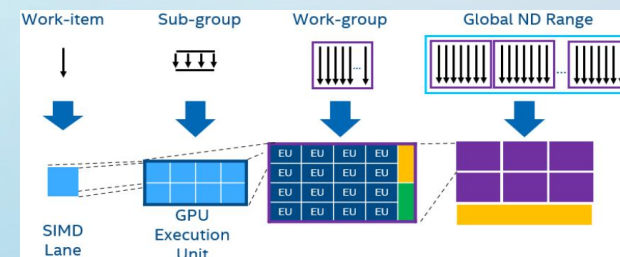
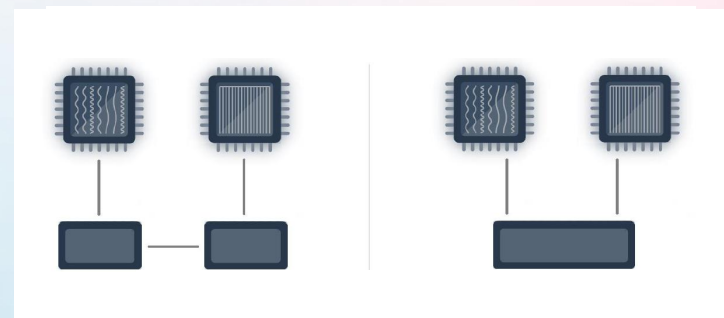
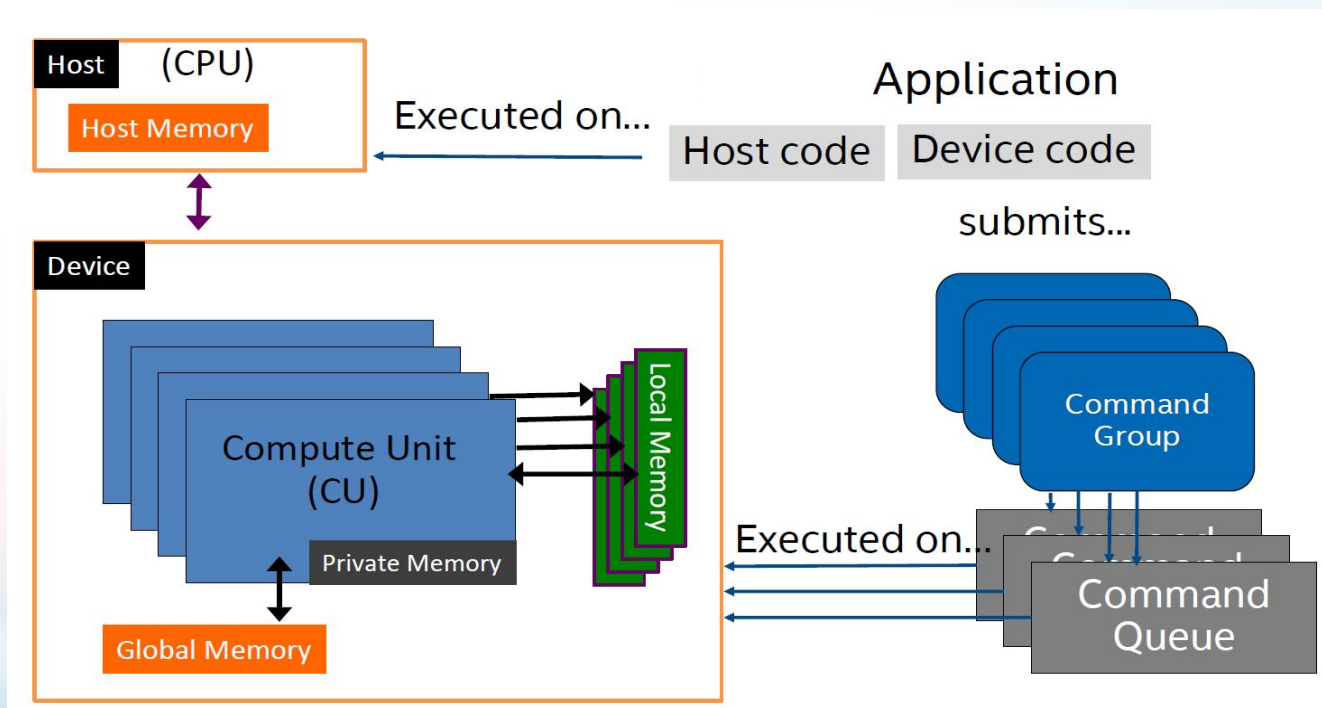
关键技术四：自适应动态运行时技术

为CUDA、ROCm等多厂商建立**统一适配接口**，实现算力资源动态注册管控；形成原生程序**跨架构动态自适应匹配解析机制**，即时映射执行。



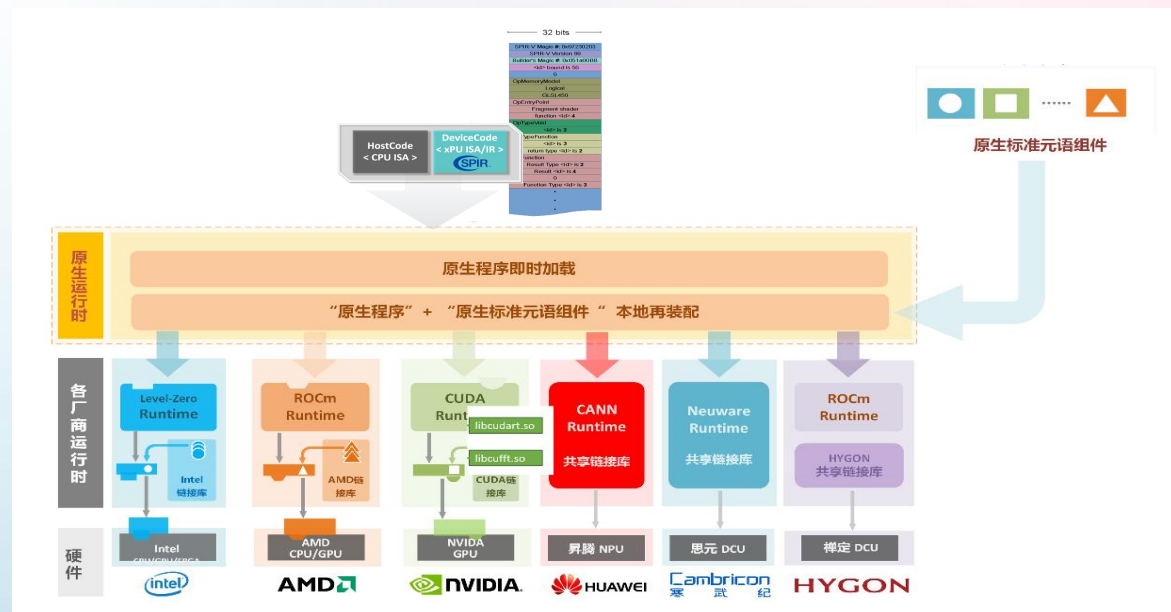
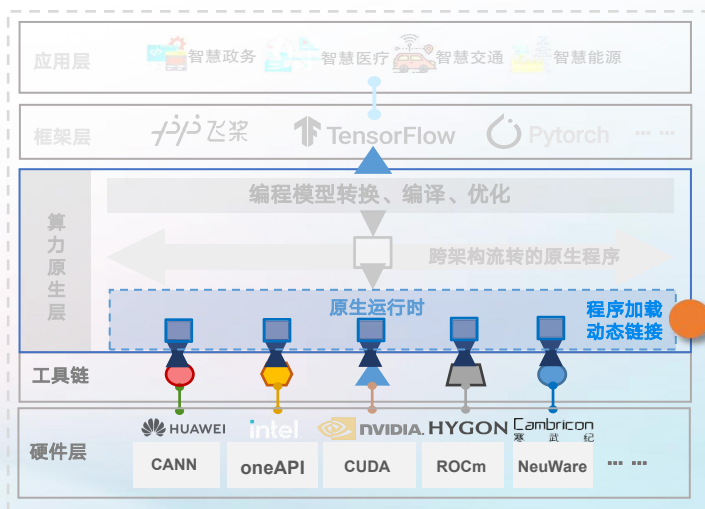
算力原生关键技术：算力抽象技术

各厂商异构加速硬件（GPU/NPU/DLA...）都是在用各自不同的方式做着同样的工作。算力原生的核心是为多类异构加速硬件建立统一设备模型、内存模型、执行模型等算力抽象，为异构资源形成统一算力设备模型与系统接口



算力原生关键技术：自适应动态运行时

以算力抽象技术为基础，在CUDA、Level-Zero、ROCm等厂商运行时之上构建虚拟算力设备及统一运行时API接口，实现多类异构系统环境、计算内核、内存等资源的动态注册管控，形成XPU虚拟设备，加载原生程序后，跨架构动态自适应匹配解析执行



原生程序跨架构动态自适应的加载、解析机制，保障计算任务与异构算力资源的即时互映射按需执行

算力原生关键技术：统一编程模型技术

建立可适用于不同异构环境的统一编程模型及范式 (基于SYCL), 对数据管理、内核调度、并行内核开发及访问提供统一的编程范式支持, 支撑实现应用一次开发; 研究与统一编程模型匹配的源源转换技术, 使其他编程模型向统一编程自动转译

```
// The CUDA kernel is a separate function
__global__ void TestKernel(int* dst) {
    auto id = blockIdx.x * blockDim.x + threadIdx.x;
    dst[id] = id;
}

int main() {
    // CUDA uses device zero by default
    cudaDeviceProp deviceProp;
    cudaGetDeviceProperties(&deviceProp, 0);
    std::cout << "Running on device: " << deviceProp.name << "\n";

    int* buffer = nullptr;
    cudaMallocHost(&buffer, count * sizeof(int));
    cudaMemset(buffer, 0, count * sizeof(int));

    TestKernel<<<count / 256, 256>>>(buffer);
    cudaDeviceSynchronize();
    // ...
}
```

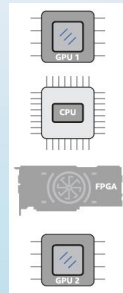
加速开发者对传统CUDA代码转换

CUDA-SYCL 源-源转换机制, 从API、内核映射等多个维度对CUDA 源代码进行多级自动转换

```
1  #include <CMCC/sycl.hpp>
2  #include <iostream>
3
4  using namespace sycl;
5
6  int main() {
7      constexpr int size=16;
8      buffer<int> B { range{ size } };
9
10     // Create queue on any available device
11     queue Q;
12     Q.submit([&](handler& h) {
13         accessor A{B, h};
14         h.parallel_for(size, [=](auto& idx) {
15             A[idx] = idx;
16         });
17     });
18     // Obtain access to buffer on the host
19     // Will wait for device kernel to execute to generate data
20     host_accessor A{B};
21     for (int i = 0; i < size; i++)
22         std::cout << "data[" << i << "] = " << A[i] << "\n";
23
24     return 0;
25 }
```

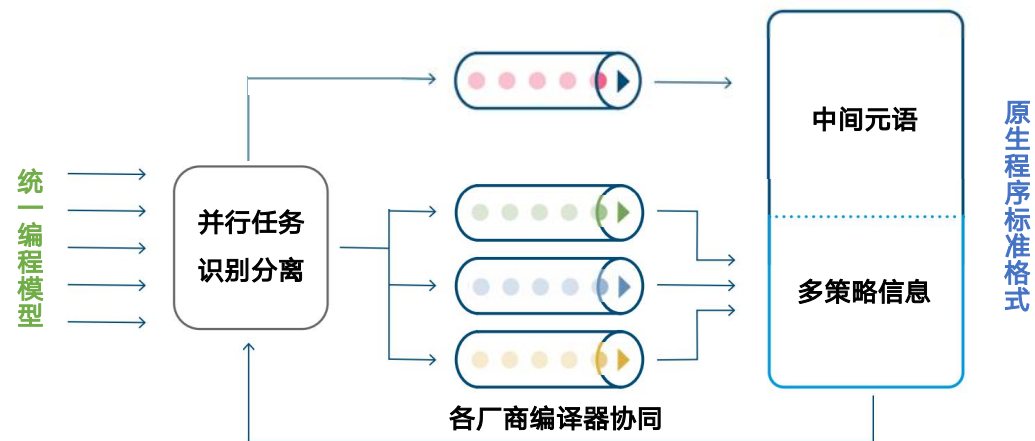
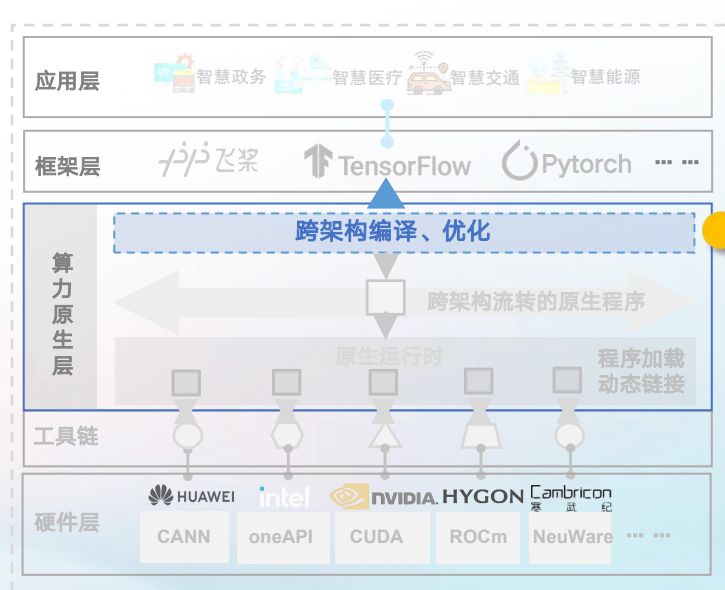
可与CUDA对标的先进“单源”统一编程模型

使开发者充分利用丰富的异构资源的同时不必考虑复杂的底层细节, 任意计算任务跨架构一次编写



算力原生关键技术：跨架构编译优化技术

构建可融入各厂商编译器的开放式编译协同机制，图、算融合编译优化方法，形成个性化执行策略生成方法，基于可跨架构互识的算力原生程序格式标准，保证跨架构的同时最大化兼顾运行性能



编译器采用LLVM、MLIR等领域标准实现开放式架构

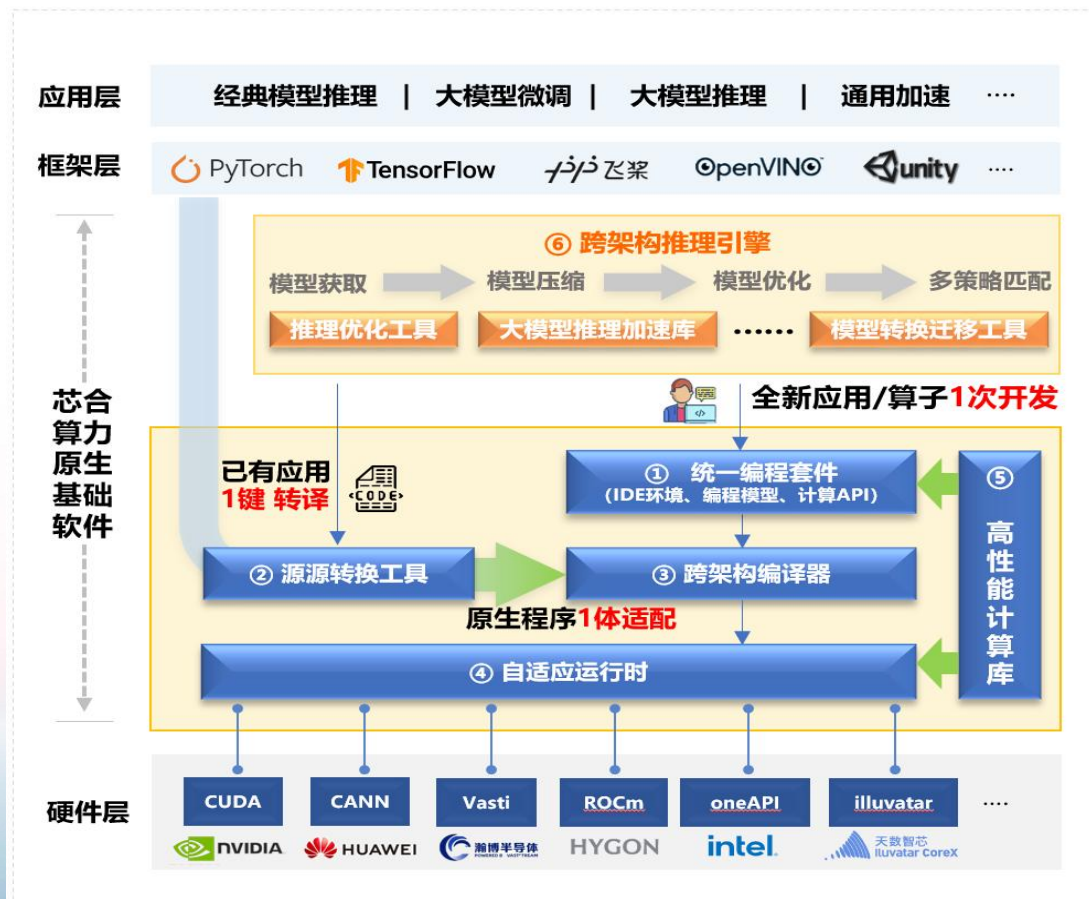
前端：解析SYCL等源码及ONNX等计算图模型

中端：进行跨架构综合优化，对并行策略等进行注入

后端：完成原生标准格式程序生成

算力原生跨架构平台——“芯合”

“芯合”跨架构基础软件系统，主要包括“统一编程套件、源源转换工具、跨架构编译器、自适应运行时、高性能计算库、跨架构推理引擎”六大通用基础功能组件，融入用户业务，支撑实现“智算应用一次封装、跨芯部署按需迁移”的任务目标



① 统一编程套件

针对 **通用程序开发** 及 **智算领域开发** 两方面需求构建传统代码编程、低代码数据流式编程两种风格的**集成编程环境**，辅助全流程开发工作。

② 源源转换工具

“CUDA → 统一编程范式”在**编程模型**及**API** 两方面的源码级转换。组件提供友好的人机交互，可帮助开发者高效开展已有工程与代码的迁移工作。

③ 跨架构编译器

CMCC编译器，便捷完成跨架构程序编译，无缝嵌入至统一编程套件，与开发流程高度匹配，高效生成多策略算力原生程序。

④ 自适应运行时

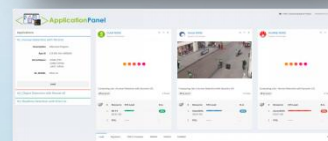
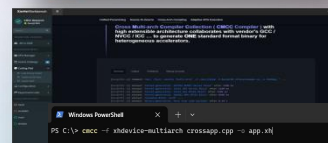
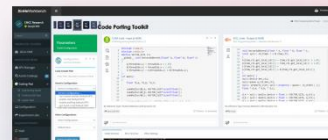
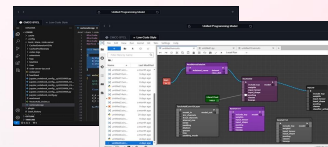
统一适配多厂商、多类型硬件系统，内存、负载、故障等异构资源情况**统一感知**、**算力原生程序快速加载部署**、**按需迁移**。

⑤ 高性能计算库

对标业界主流DNN、BLAS等常用计算API的**高性能计算库**，最大化实现用户自定义应用的迁移率，同时满足系统跨架构运行性能保障需要。

⑥ 跨架构推理引擎

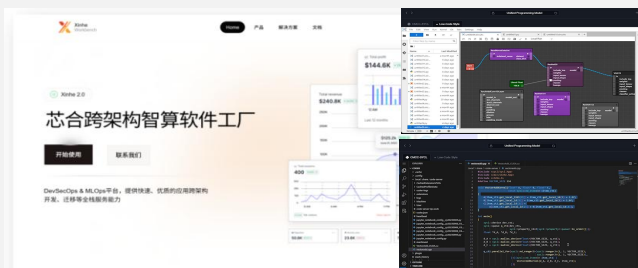
运用**XPU抽象高动态感知管理显存、通信等物理资源**，跨架构模型量化切分、算子融合、显存优化，软硬协同挖潜，极致提升系统吞吐及推理效率。



算力原生跨架构平台—“芯合”

“芯合”系统已实现7家厂商AI芯片一体适配，具备应用跨架构一键迁移、模型高效率一键部署功能，当前已用于支撑4省市应用国产化迁移及大模型异构部署

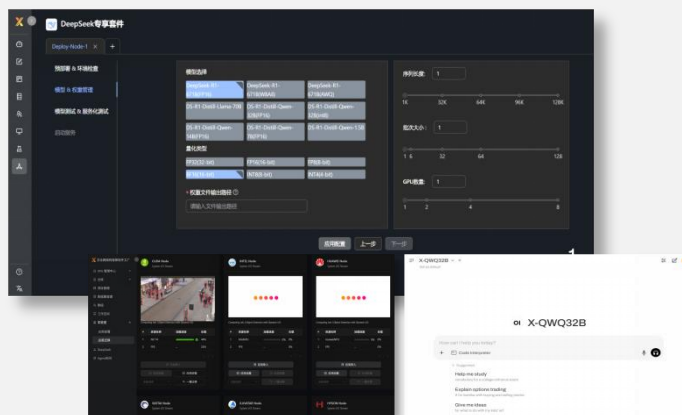
智算应用一次开发、跨7家厂商一键迁移



应用迁移对象	A—B	B—A	A—C	C—A	B—C	C—B
应用1 ResNet+算法迁移时间	15.298s	16.609s	13.298s	16.218s	18.006s	17.226s
应用2 YOLO+算法迁移时间	11.698s	12.206s	9.298s	12.602s	14.006s	12.126s

- 7家厂商芯片插件化接入，适配时间<1月
- CUDA应用源码自动转译，转译率>95%
- 应用一次开发迁移运行，热迁移时间<20秒

多规格模型及智算应用一键部署



- 实时扫描异构硬件后端，根据模型特征与规模匹配算力资源，实现自动参数配置及流程化部署
- DeepSeek在三家异构硬件上一键部署，提效7.5倍

赋能4省市应用国产化部署迁移



- 在山东、天津、福建和广西4个省公司试点应用，赋能路桥巡检等多场景智算应用国产化部署迁移
- 支撑13家用户近百套大模型在异构环境下部署

“芯合”开源进展及计划

“芯合”系统已在OIF CFN WG开放跨架构编译器、自适应运行时，并开源英伟达后端适配源码，后续将进一步完善开发者指南、开放统一编程模型，面向应用开发者提供跨架构开发及运行体验

芯合原生平台

1 原生编译器部署

1.1 基础需求

- 硬件：X86通用服务器一台，CPU > 4、内存 > 8G
- 操作系统：Linux（推荐使用ubuntu20.04 或 ubuntu22.04）

1.2 宿主机配置

- docker安装（推荐使用24以上版本）

参考：<https://docs.docker.com/engine/install/ubuntu/>

1.3 镜像获取

移动云盘：链接<https://cayun.139.com/m/7125CloXzkhpXi> 提取码

1.4 编译器容器部署

通过 docker 命令加载本地下载好的编译器镜像。

```
sudo docker load --input xh-cmcc-compiler-v1.0.tar
```

其中，xh-cmcc-compiler-v1.0.tar 为芯合编译器镜像文件的名称。

2.1 硬件支持

芯合原生运行时当前支持以下3种加速硬件：

- 英伟达：tesla V100 和 tesla T4
- 华为：ansend 910
- 海思：VA1A

2.2 瀚博环境运行时部署（VA1A）

2.2.1 基础需求

- 硬件：具备瀚博VA1A加速卡的X86架构服务器一台，CPU > 8、内存 > 16G（推荐）
- 操作系统：Linux（推荐使用ubuntu20.04）
- 内核版本：3.99 及以上

2.2.2 宿主机配置

(1) 安装环境依赖包：

```
yum install cairo-devel.x86_64
yum install libdrm-devel.x86_64
yum install libm-dev.x86_64
yum install libm-dev.x86_64
yum install libm-dev.x86_64
yum install libm-dev.x86_64
yum install libm-dev.x86_64
yum install libm-dev.x86_64
yum install libm-dev.x86_64
yum install libm-dev.x86_64
yum install libm-dev.x86_64
yum install libm-dev.x86_64
```

- OIF Repository: <https://opendev.org/cfn/computing-native>
- 跨架构编译器与自适应运行时

自适应运行时接口

- OIF Link: https://opendev.org/cfn/computing-native/src/branch/master/src/XINHE_Runtime/include/HYPERCROSS/hycr_runtime_api.h
- 提供设备管理、内存管理、任务管理、平台管理等相关标准化接口

英伟达后端适配

- OIF Link: https://opendev.org/cfn/computing-native/src/branch/master/src/XINHE_Runtime/src/runtime/DeviceCUDA.cpp
- 提供异构后端接入自适应运行时代码样例



谢谢聆听